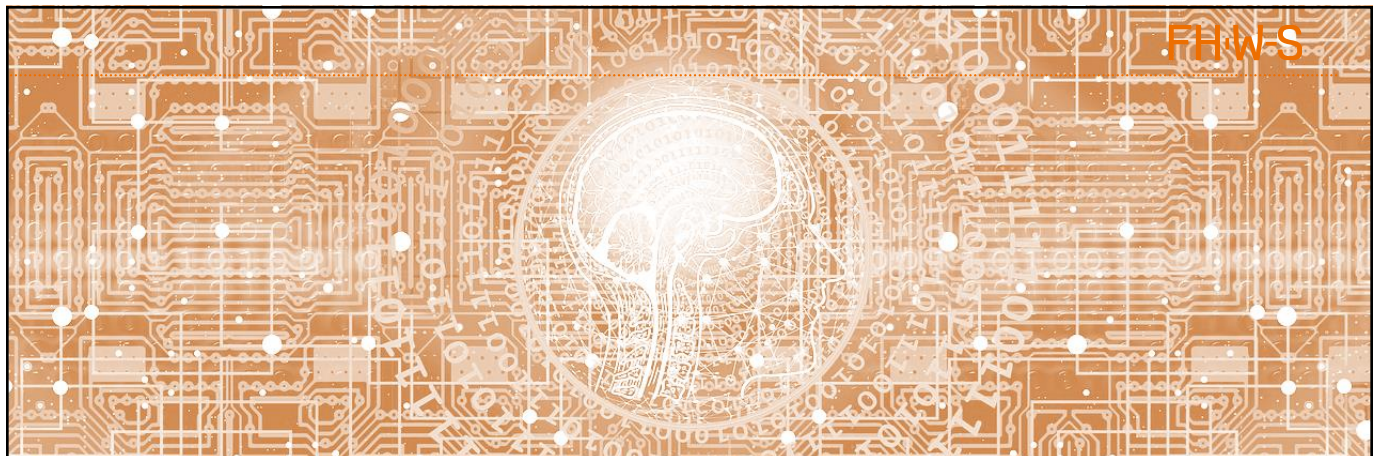




Software Engineering and Cyber Security

Prof. Dr. Marian Daun

FHWS University of Applied Sciences
Würzburg-Schweinfurt

The slide has a background image of a desk with a notebook, a pen, and a smartphone. A semi-transparent white box with an orange border contains the agenda items. The FHWS logo is in the bottom left, and the number '2' is in the bottom right.

Agenda

- Software Engineering Fundamentals
- Requirements Engineering
- Architecture Design
- Modelling
- Development and Implementation
- Software Quality Assurance

FHWS University of Applied Sciences
Würzburg-Schweinfurt

2

Modelling

Part C: Function



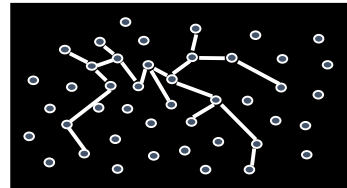
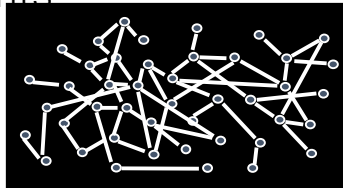
Group Discussion

What is the Difference between Structure and Function?

Introduction to Use Cases

Why Use Cases?

- Use cases structure complex processes with multitudes of possible sequences of actions.
- One use case shows a single sequence of actions to reduce complexity.
- Hence, use cases support the understandability of requirements



Use Case

D A use case is a specific way of using the system by performing some part of the functionality. Each use case constitutes a complete course of events initiated by an actor and it specifies the interaction that takes place between an actor and the system. A use case is thus a special sequence of related transactions performed by an actor and the system in dialogue. The collected use cases specify all the existing ways of using the system.

Use Case Diagrams

Use Case Diagrams – Overview

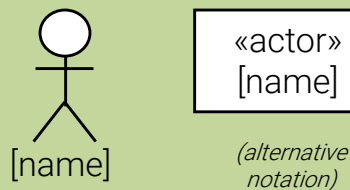
- Use case diagrams represent the
 - relevant relations between external actors (such as users) and the use case
 - and relations between use cases.
- A single use case diagram does not represent the involved use cases at a sufficient level of detail!
 - A more detailed specification of the involved use cases is required.

Actor

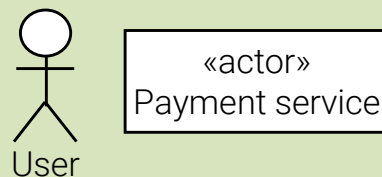
Definition

- Covers external entities that interact with the system.
- Abstracts from specific instances by describing roles.
- Interacts with the system by participating in use cases.

Notation



Example

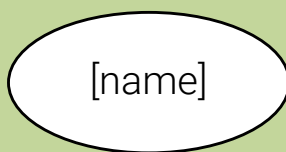


Use Case

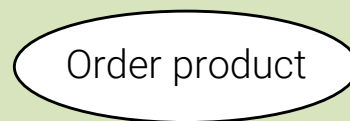
Definition

- A specific way of using the system by performing some part of the functionality.
- Constitutes a complete course of events initiated by an actor.
 - Specifies the interaction that takes place between an actor and the system.
- Collected use cases specify all the existing ways of using the system.

Notation



Example

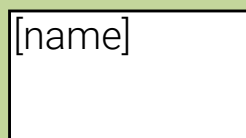


System Boundary

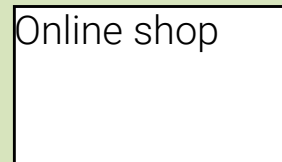
Definition

- Separates the system from its operational context.
- Use cases are placed inside the system boundary while external actors are placed outside.
- The system boundary defines the scope for which the requirements are specified.

Notation



Example

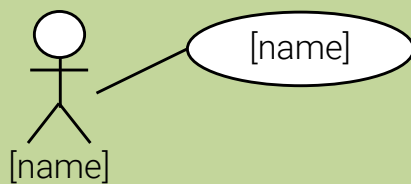


Association

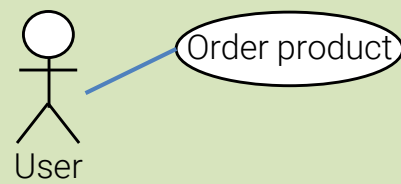
Definition

- Represents the participation of an actor in a use case.
- Instances of actors (i.e. instances of external entities having the respective role) can communicate with instances of use cases.
- Association relations are bidirectional.

Notation



Example

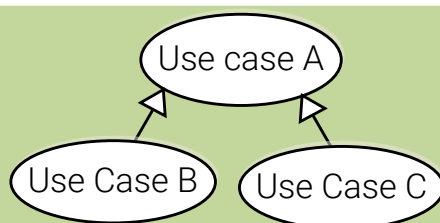


Use Case Generalization

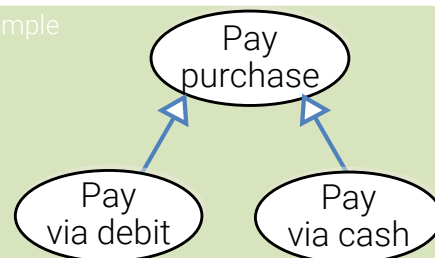
Definition

- Specializes a more general use case into one or multiple specialized, i.e. more specific use cases.
- Specialized use cases may have additional properties and associations.

Notation



Example

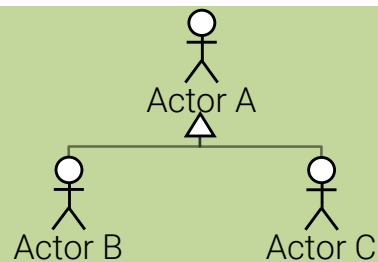


Actor Generalization

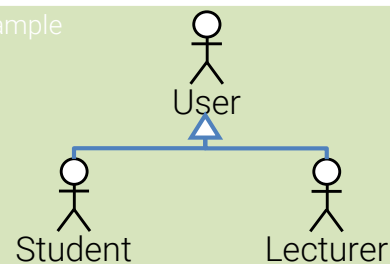
Definition

- Specializes a more **general actor into** one or multiple **specialized**, i.e. more specific actors.
- Specialized actors may have **additional properties** and associations.

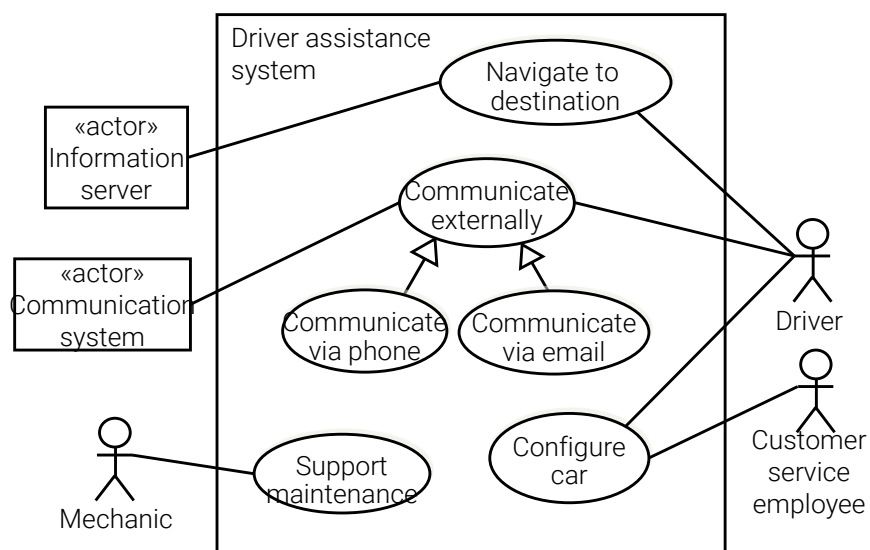
Notation



Example



Use Case Diagram

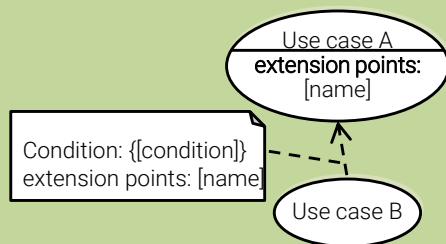


Extend Relationship

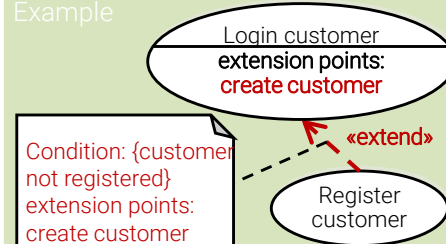
Definition

- Extend relationship allows to extend a use case with **additional functionality**, i.e. the corresponding sequence of interactions is conditionally executed.
- **Extending** use cases can be used **in different extended** use cases.
- An extend relationship consists of a **condition** and a reference to an **extension point** within the extended use case.

Notation



Example

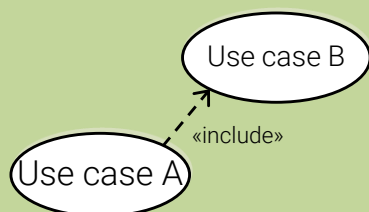


Include Relationship

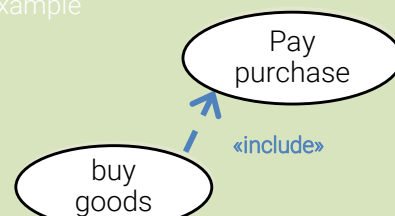
Definition

- An include relationship expresses that **one use case is executed when another use case is executed**.
- Use case A **includes in any case** the behaviour of use case B.
- Include relationships are typically used when **a sequence of interactions is part of more than one use cases** of the system.

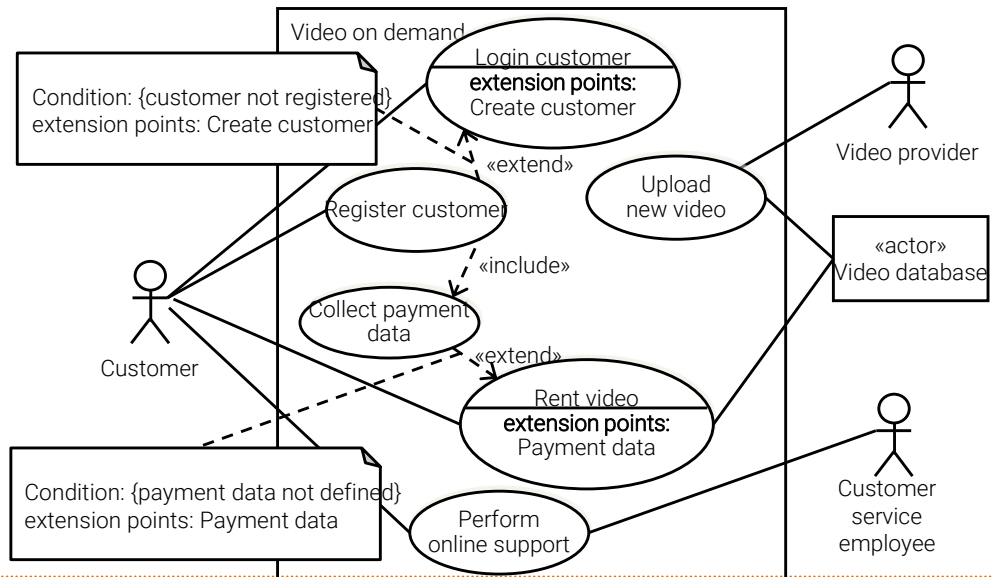
Notation



Example



Use Case Diagram



Exercise

Create a Use Case Diagram for a Webshop

Use Case Templates

Overview

- Use case templates provide [detailed descriptions](#) for the use cases defined in a use case diagram.
- Use case templates shall be [specifically designed](#) for each companies specific purposes.
- Common use case [reference templates](#) provide a good [starting point](#) for the individual customization.

Background information on templates in general:

- Templates are based on [expert knowledge](#).
- A template defines [relevant](#) types of [information](#).
- A template [structures](#) the information.

Categories

- Use case templates provide slots to define information in different categories.
- Commonly used [categories](#) are:
 - Use case [management](#) information
 - Use case [diagram](#) information
 - [Contextual](#) information
 - [Scenario](#) information
 - [References](#) to other models and diagrams
- Use case templates can be defined on different [levels of detail](#):
 - High-level use case templates
 - Detailed use case templates

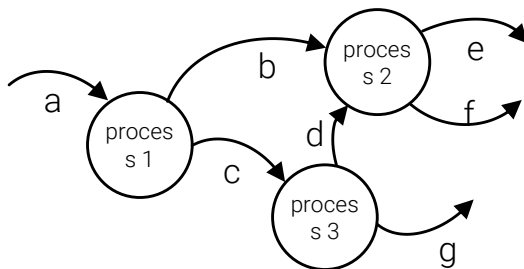
High-Level Use Case Template

No.		Section	Content / Explanation
ID	1.2	Name	Unique name for the use case.
Management	2.1	Author(s)	Name of the authors of the use case description.
Context	3.1	Source	Source from where the use case stems.
Use Case Definition	4.2	Short description	Concise description of the use case.
	4.4	Goal(s)	Goal(s) that shall be satisfied by executing the use case.
	4.5/ 4.6	Actor(s)	Enumeration of all actors involved in the use case.
	4.7	Pre-condition(s)	Prerequisites to be fulfilled before use case execution.
	4.8	Post-condition(s)	A list of conditions that hold after execution of the use case.
Relationships	5.2	Relationship to other use cases	Short description of the relations to other use cases (apply only if these relationships are not documented by a use case diagram.

Data Flow vs. Control Flow

Properties of Data Flows

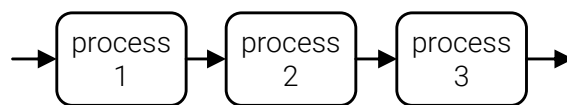
- Describe pipelines between processes. (see [DeMarco 1979])
- Pipelines transmit packages of information of known composition. (see [DeMarco 1979])
- Packages of information contain material or immaterial objects (e.g. sheets of paper, payment information).



No explicit information about process sequences; all process can, in principle, be active at the same time.

Properties of Control Flows

- Defines process execution sequences, as well as events and conditions under which processes are executed.
- Symbolizes the passing of a trigger from one activity to the next.
- **Note:** A control flow typically includes a flow of information, e.g., an event includes its source and time of occurrence.



Only one process can be active at one point in time, e.g., process 2 can execute only if process 1 is completed.

Data Flow Diagrams (DFDs)

Data Flow

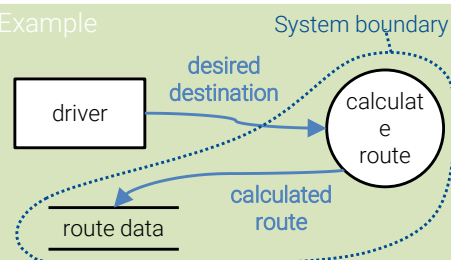
Definition Data Flows:

- Describe the transportation of information packages of known composition. Information packages may contain material or immaterial objects (e.g., information, products, energy, etc.).
- Can be defined between:
 - Two processes.
 - A process and a data store.
 - A source/sink and a process.

Notation



Example



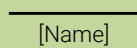
Data Store

Definition

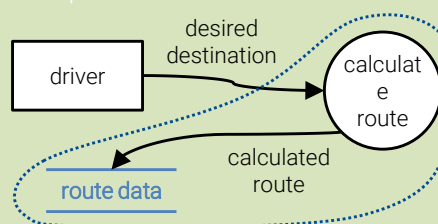
A Data Store:

- Defines a (physical or technical) repository of data (e.g., files, folder, etc.).
- Contains "data at rest".
- Persistently records data produced by the system itself or received from the context.
- A process can access or retrieve the data in a data store.

Notation



Example



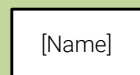
Source and Sink

Definition

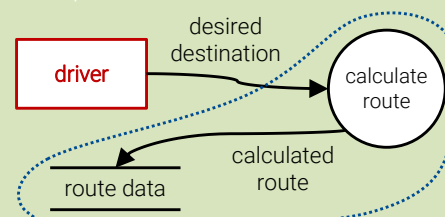
Sources and Sinks:

- Are external objects outside of the system boundary (e.g., persons, organizations, other systems), i.e. sources and sinks reside in the context of the system.
- Are sender of information packages to the system and/or receiver of information packages from the system.

Notation



Example



Exercise

Model a Data Flow Diagram for an Adaptive Cruise Control

Structured Analysis

Motivation for Structured Analysis

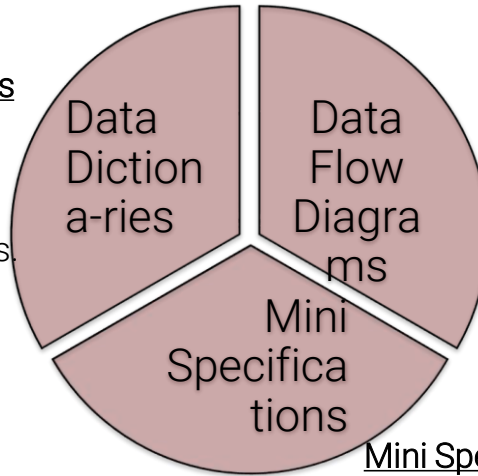
- "Analysis is the study of a problem prior to taking some action."
- Primary outcome of a Structured Analysis is a structured specification document which
 - is highly maintainable,
 - reduces complexity by means of effective partitioning techniques,
 - uses graphical representations rather than narrative text.

[DeMarco 1979]

Three Components

Data Dictionaries

define the composition of the data in data stores and flows.



Data Flow Diagrams (DFD)

define processes and data flows between processes and sources/sinks.

Mini Specifications

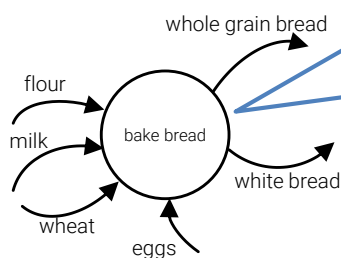
define primitive functions.

Data Dictionaries

Motivation

Data flow diagrams can be ambiguous, i.e., they can be interpreted differently

by different stakeholders and/or
at different points in time.



Ambiguous process "bake bread":

- What is the relationship between the inputs and the outputs?
- When is which output produced?
- Are all inputs needed for both outputs?
 - If so, what is the difference between the outputs?
 - If not, which output requires which input?

Definition

D A data dictionary defines the structure of each data flow and data store in a Data Flow Diagram.

- The entries of a data dictionary are typically, defined in an EBNF (extended Backus-Naur Form) language.
- In the following we outline a language for defining entries of a data dictionary.

Equivalence Operator, AND-Operator

Definition

- “+” means “and”.
Composition of a data elements from other data elements.
- “=” means “is equivalent to”.
Definition of a data element in terms of other data elements.

Notation

+

=

Example

$$x = a + b$$

x is equivalent to the
composition of a and b

name = given name + family name

Optional Operator

Definition

- “(..)” means “optional”.
Zero or one of the elements or expressions in brackets.

Notation

(..)

Example

$$x = a + b (+ c)$$

x is equivalent to the
composition of a and b, and
in some cases to the
composition of a and b and c

address = street + city (+ country)

Selection Operator

Definition • „[...]“ means “either – or”.

- Choice of exactly one of several possible data elements.
- Exclusive choice options are separated by “|”.

Notation

[...|...|...]

Example

$x = a + [b | c]$

x is equivalent to the composition of
a and (either b or c)

sandwich = bread + [cheese | meat]

Iteration Operator (1)

Definition • „{...|...|...}“ means “selection of”.

- Repetitions of the data element in {...}.

Notation

{...|...|...}

Example

$x = \{a | b | c\}$

x is equivalent to
selection (0 to N with N=no. of elements)
of the data elements a, b and c

pet = {dog | cat | bird}

Iteration Operator (2)

- Definition
- Number of repetition of data elements can be constrained with lower (x) and upper boundaries (y).
 - Lower boundary can be defined without an upper boundary and vice versa.

Notation

$$x\{\dots|\dots|\dots\}y$$

Example

$$x = 1\{a | b | c\}2$$

x is equivalent to **at least one** but **at most two** selections of the data elements **a, b and c**

$$\text{pet} = 1\{\text{dog} | \text{cat} | \text{bird}\}2$$

Data Primitive

- Definition
- Primitive data, which is not further decomposed, is defined by using `""`.
 - Typically defines well understood or atomic information packages.

Notation

$$""$$

Example

$$x = ""$$

The terms **"0"** is **well-known** and **not further defined or refined**

$$\text{Drinks} = \{""\text{water}"" | ""\text{mango juice}"" | \dots\}$$

Comment

Definition • Textual description or explanation.

- Can be used in case the data is well understood, but not necessarily an atomic information packages.
- Can be used to reference where the data is defined (e.g., reference to another supplementary document).

Notation

* ... *

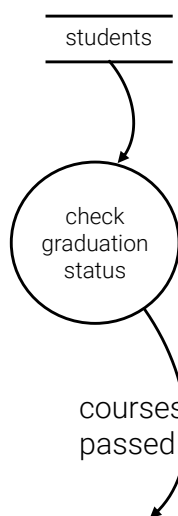
Example

$x = a + b$ *description*

description is a
comment

name = given name + family name
family name always written last

Example



```

...
students      = {student}
student      = studentID
              + degree program
              + degree progress
              + student profile
degree program = degree name
              + {course to take + course description}
degree progress = {course passed} + {course failed}
student profile = name
                + {address}
                + date of registration
                *student since ...*
address        = street + zip + city + country + address
type
address type   = ["home" | "parents" | "emergency contact"]
...
  
```

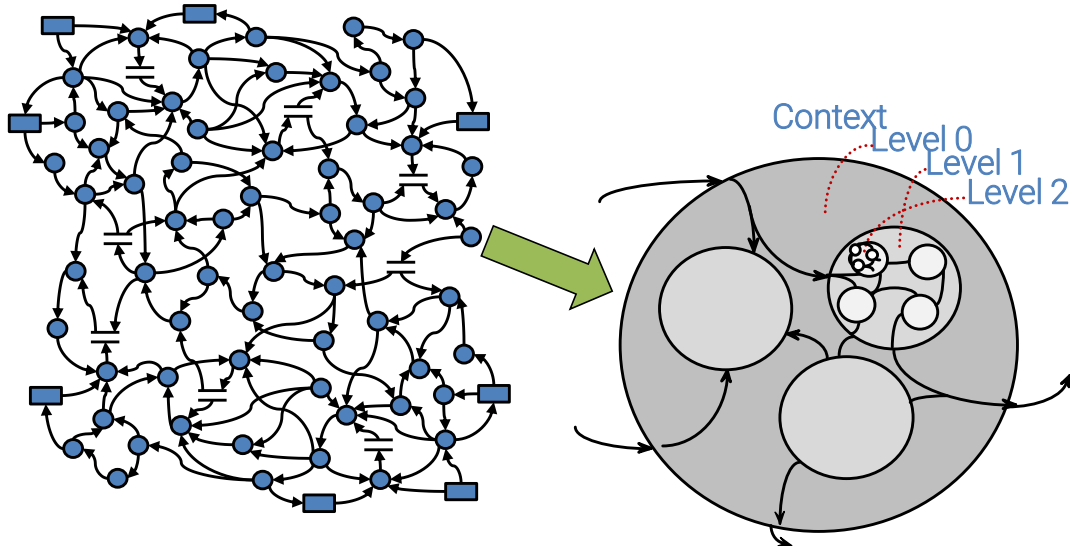
Hints

- Avoid redundancies.
- Reuse already defined data elements, where possible.
- Aim to adopt terms known to the stakeholders.
- Refine data element definition only if it is necessary to achieve a better understanding.
- Avoid circular definitions of data elements.
- Define synonyms to solve naming conflicts.
- Stop defining terms further when your user clearly understands their meaning.

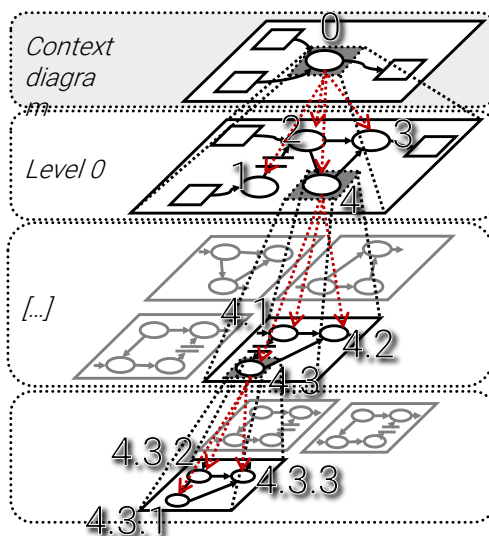
based on [DeMarco 1979]

Hierarchization of Data Flow Diagrams

Reduction of Complexity: Levelling



Data Flow Diagrams in Structured Analysis



- **Reduction** of a complex problem into a set of smaller ones that can be **analysed separately**.
- Data flow diagrams serve as the main tool for **partitioning the system** on different layers of abstraction.
- **Level of detail** increases on lower levels of the DFD hierarchy.
- Supports **top-down analysis**, i.e., the successive partitioning until **functional primitives** are identified.
- Supports **bottom-up analysis**, i.e., the successive **composition** of functional primitives to a **common superstructure**.

Balancing of Data Flow Diagrams

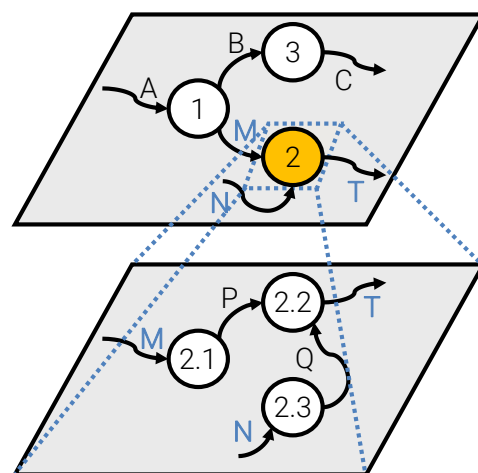
Strict balancing rules for assuring consistency between different levels of DFDs in the DFD hierarchy.

Three types of balancing:

- Visible balancing
- Data dictionary balancing
- Data store balancing

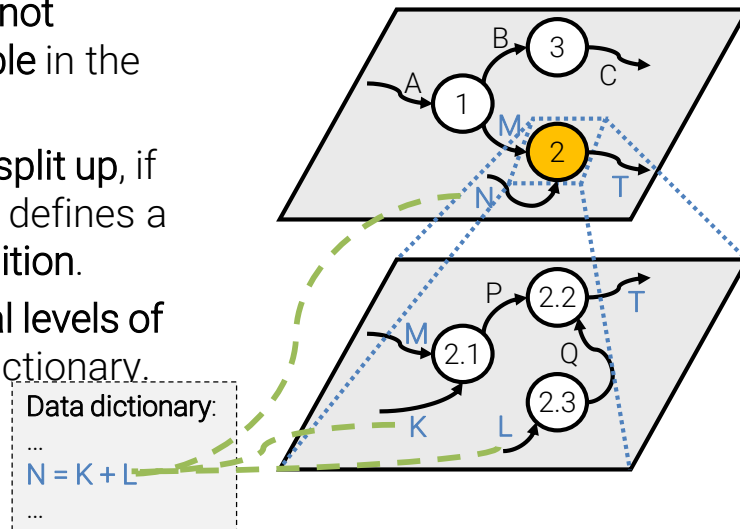
Visible Balancing

- Every input and output data flow of the parent process node **must** also be directly visible in the child data flow diagram.
- Consistency** is immediately visible and can be checked easily.



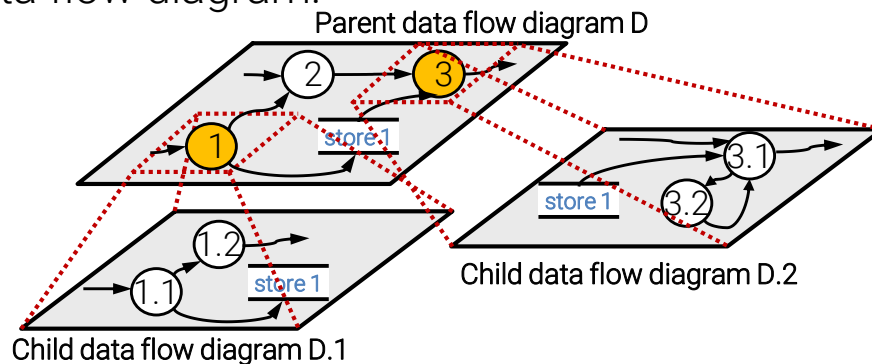
Data Dictionary Balancing

- The balance must **not necessarily be visible** in the data flow diagram.
- **Data flows** can be **split up**, if the data dictionary defines a **respective composition**.
- May involve **several levels of detail** in the data dictionary.

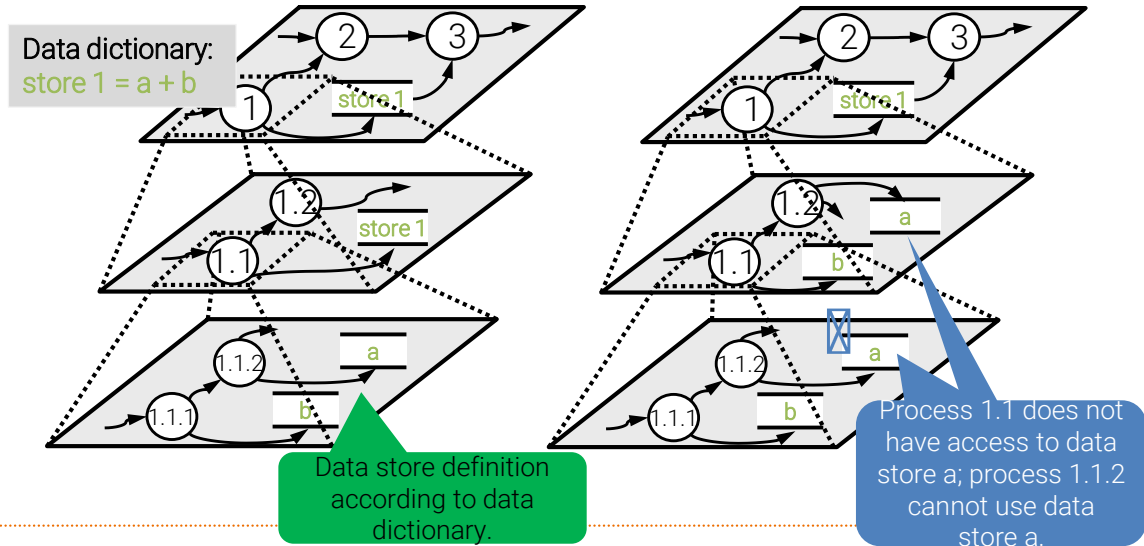


Visual Data Store Balancing

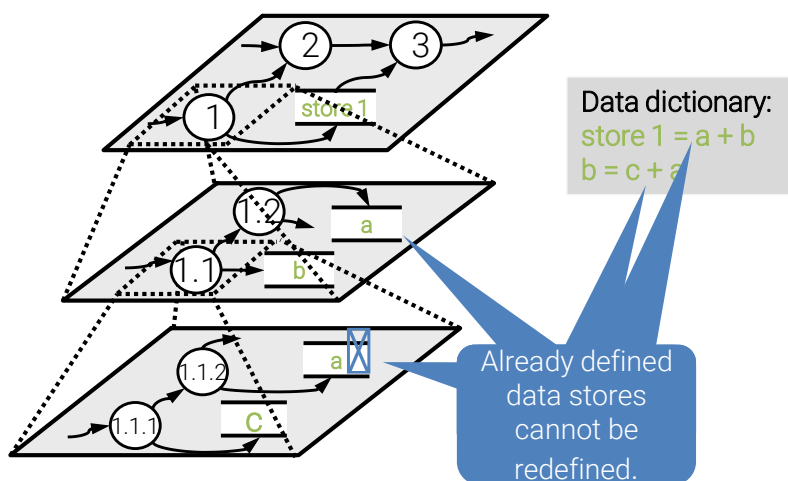
All child data flow diagrams have to define all read and write accesses for each data store defined at their parent data flow diagram.



Data Dictionary Data Store Balancing

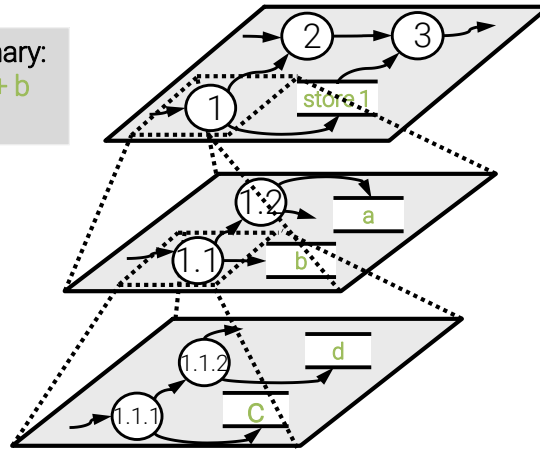


Data Dictionary Data Store Balancing



Data Dictionary Data Store Balancing

Data dictionary:
store 1 = a + b
b = c + d



Exercise

Restructure the Data Flow Diagram for the Adaptive Cruise Control and use Balancing Rules

Mini Specifications

Properties

A Mini Specification

- describes how a primitive process (function) produces its outputs based on its inputs in terms of a coarse strategy without defining a concrete algorithm.
- is typically defined using natural language.
- typically defines the functionality of the functional primitive as a sequence of steps which use inputs to produce the outputs.

[DeMarco 1979]

Questions for Self-Assessment

What is a use case?
 What is a use case diagram and what are its elements?
 What are the three key components of Structured Analysis?
 How can the context of the system be identified in DFDs?
 How can complexity be handled in DFD?
 What is the purpose of mini specifications?
 Explain the relation of data dictionaries and the balancing rules!

Thanks

This Lecture Material is based on Lecture Material provided by
Prof. Dr. Klaus Pohl from **University of Duisburg-Essen**

The original Lecture Material was created by:

Prof. Dr. Klaus Pohl
 Torsten Bandyszak
 Jennifer Brings
 Johanna Buchner

Prof. Dr. Marian Daun
 Felix Föcker
 Andrea Salmon
 Philipp Schmidt

Prof. Dr. Bastian Tenbergen
 Prof. Dr. Thorsten Weyer